Example:
Test Case 1: Login Test

Code

```
import os
import pathlib
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options

# Paths and configuration
BASE_DIR = pathlib.Path(__file__).resolve().parent.parent
SCREENSHOT_DIR = BASE_DIR / "test-results" / "screenshots"
SCREENSHOT_DIR.mkdir(parents=True, exist_ok=True)

# Test Configuration - configurable via environment variables
BASE_URL = os.environ.get("BASE_URL", "http://localhost:3000")
LOGIN_URL = f"{BASE_URL}/login"
TEST_USERNAME = os.environ.get("TEST_USERNAME", "test_admin")
TEST_PASSWORD = os.environ.get("TEST_PASSWORD", "TestAdmin@2026")
HEADLESS_MODE = os.environ.get("HEADLESS", "true").lower() in ("true", "1", "yes")


class TestLogin:
    """
    Test Case 1: ProcuraMed User Login Automation Test Suite
    Covers both positive authentication and negative validation scenarios.
    """

    def setup_method(self, method):
        """Setup Chrome WebDriver before each test execution."""
        options = Options()
        if HEADLESS_MODE:
            options.add_argument("--headless=new")
        options.add_argument("--window-size=1920,1080")
        options.add_argument("--no-sandbox")
        options.add_argument("--disable-dev-shm-usage")
        options.add_argument("--disable-gpu")
        options.add_argument("--ignore-certificate-errors")

        self.driver = webdriver.Chrome(options=options)
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 12)

    def teardown_method(self, method):
        """Tear down and close the WebDriver session after each test."""
        if hasattr(self, "driver") and self.driver:
            self.driver.quit()

    def click_button(self, element):
        """Helper to reliably click button across animation frames."""
        self.driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", element)
        try:
            element.click()
        except Exception:
            self.driver.execute_script("arguments[0].click();", element)

    def test_valid_user_login(self):
```

```
        """
        Test Step: Navigate to login page, enter valid username & password,
        submit form, and verify successful redirection to Admin Dashboard.
        """
        driver = self.driver
        wait = self.wait

        # Step 1: Navigate to the login page
        driver.get(LOGIN_URL)
        username_field = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
        driver.save_screenshot(str(SCREENSHOT_DIR / "step1_login_page_displayed.png"))
        assert "Sign In to ProcuraMed" in driver.page_source, "Login page failed to load"

        # Step 2: Enter valid username
        username_field.clear()
        username_field.send_keys(TEST_USERNAME)

        # Step 3: Enter valid password
        password_field = wait.until(EC.visibility_of_element_located((By.NAME, "password")))
        password_field.clear()
        password_field.send_keys(TEST_PASSWORD)

        driver.save_screenshot(str(SCREENSHOT_DIR / "step2_credentials_entered.png"))

        # Step 4: Click on login button
        login_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']")))
        self.click_button(login_btn)

        # Step 5: Verify successful login and landing on Admin Dashboard
        wait.until(EC.url_contains("/admin/dashboard"))
        dashboard_heading = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//h1[contains(text(), 'System Administrator Dashboard')]")
        )

        driver.save_screenshot(str(SCREENSHOT_DIR / "step3_admin_dashboard_success.png"))

        assert "/admin/dashboard" in driver.current_url, f"Expected dashboard URL, got: {driver.current_url}"
        assert dashboard_heading.is_displayed(), "System Administrator Dashboard heading not visible"

    def test_invalid_password_error(self):
        """
        Negative Test: Attempt login with valid username but incorrect password.
        Verify that an appropriate error alert message is displayed.
        """
        driver = self.driver
        wait = self.wait

        driver.get(LOGIN_URL)
        username_field = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
        username_field.clear()
        username_field.send_keys(TEST_USERNAME)

        password_field = wait.until(EC.visibility_of_element_located((By.NAME, "password")))
        password_field.clear()
        password_field.send_keys("WrongPassword@999")

        login_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']")))
        self.click_button(login_btn)

        # Verify error alert appearance
        error_alert = wait.until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "div.bg-rose-50"))
        )
        driver.save_screenshot(str(SCREENSHOT_DIR / "step4_invalid_password_error.png"))
```

```
        assert "Incorrect password. Please try again." in error_alert.text, (
            f"Expected password error message, found: {error_alert.text}"
        )

    def test_nonexistent_user_error(self):
        """
        Negative Test: Attempt login with a non-existent username.
        Verify that 'No account found with the given username.' is displayed.
        """
        driver = self.driver
        wait = self.wait

        driver.get(LOGIN_URL)
        username_field = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
        username_field.clear()
        username_field.send_keys("unknown_user_9999")

        password_field = wait.until(EC.visibility_of_element_located((By.NAME, "password")))
        password_field.clear()
        password_field.send_keys("RandomPass@123")

        login_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']")))
        self.click_button(login_btn)

        error_alert = wait.until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "div.bg-rose-50"))
        )
        driver.save_screenshot(str(SCREENSHOT_DIR / "step5_nonexistent_user_error.png"))

        assert "No account found with the given username." in error_alert.text, (
            f"Expected user not found error message, found: {error_alert.text}"
        )
```

Screenshot

```
PS F:\ProcuraMed> pytest tests/test_login.py -v
============================= test session starts =============================
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.7.0
rootdir: F:\ProcuraMed
collected 3 items

tests/test_login.py::TestLogin::test_valid_user_login PASSED            [ 33%]
tests/test_login.py::TestLogin::test_invalid_password_error PASSED      [ 66%]
tests/test_login.py::TestLogin::test_nonexistent_user_error PASSED      [100%]

============================== 3 passed in 18.05s ==============================
PS F:\ProcuraMed>
```

Figure 1 : Login Test successfully (Pytest Execution Results)

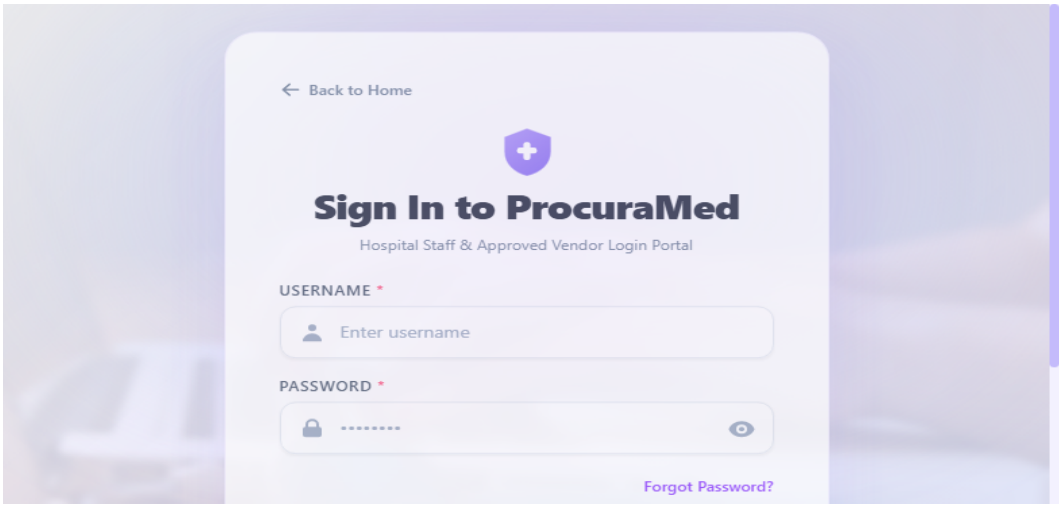


Figure 2 : ProcuraMed Login Page

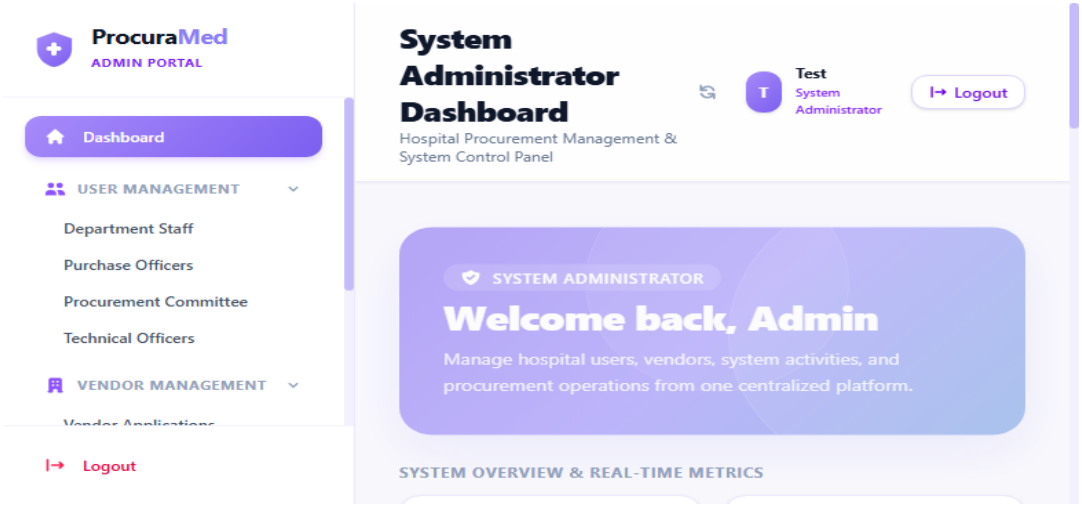


Figure 3 : System Administrator Dashboard after Successful Login

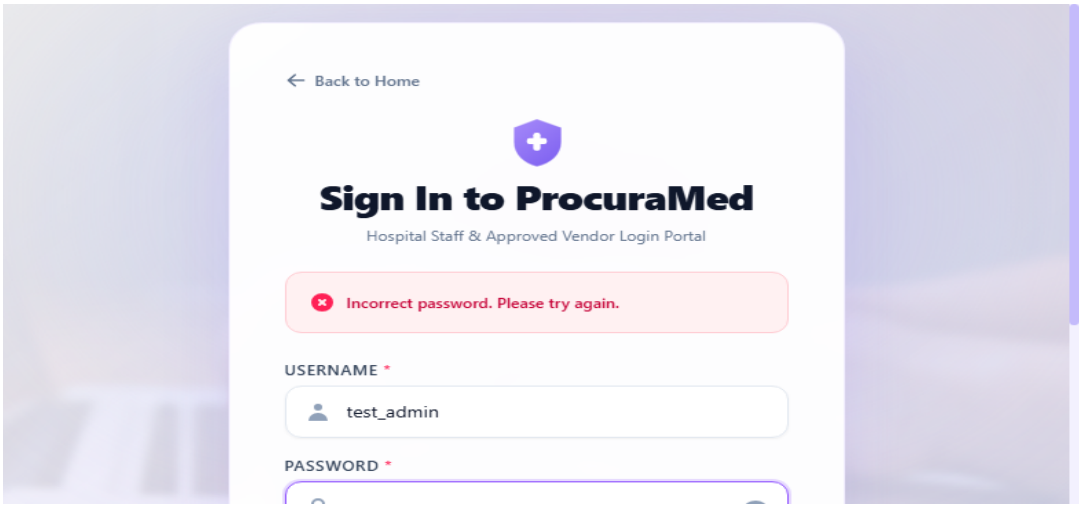


Figure 4 : Login Validation Error Message on Invalid Password

Test Report
Test Case 1

Project Name: ProcuraMed					
Login Test Case					
Test Case ID: Test_1			**Test Designed By:** QA Automation Engineer		
Test Priority(Low/Medium/High): High			**Test Designed Date:** 21/10/2026		
Module Name: Login page			**Test Executed By :** QA Automation Engineer		
Test Title : verify login with valid username and password.			**Test Execution Date:** 21/10/2026		
Description: User has valid username and password.					
Pre-Condition : User has valid username and password					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Navigate to the login page		Display login page	Login page displayed	**pass**
2	Valid username entered	Username: test_admin	Admin should enter the main page	Admin entered to the main page	**pass**
3	Valid password entered	Password: [Configured via ENV]			
4	Click on login button				
Post-Condition: Username and password is checked with database values for successful login					

Test Report – Negative Test Cases
Test Case 1.1: Invalid Credentials & Security Checks

Project Name: ProcuraMed					
Login Negative Validation Test Cases					
Test Case ID: Test_1_Neg			**Test Designed By:** QA Automation Engineer		
Test Priority(Low/Medium/High): Medium			**Test Designed Date:** 21/10/2026		
Module Name: Login page			**Test Executed By :** QA Automation Engineer		
Test Title : verify login with invalid credentials displays error message.			**Test Execution Date:** 21/10/2026		
Description: Verify authentication rejections and proper error messages on invalid input.					
Pre-Condition : User is on the ProcuraMed portal login page					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Valid username with incorrect password entered	Username: test_admin Password: WrongPassword@999	Display error: 'Incorrect password. Please try again.'	Error alert displayed: 'Incorrect password. Please try again.'	**pass**
2	Unregistered username entered	Username: unknown_user_9999 Password: RandomPass@123	Display error: 'No account found with the given username.'	Error alert displayed: 'No account found with the given username.'	**pass**
Post-Condition: System rejects unauthenticated access; no JWT token is issued.					